



100x Data Engineer — Week 0

Development Environment & Database Setup

Before starting the 100x Data Engineer roadmap, I set up my local development environment and prepared the PostgreSQL database that I will use throughout the roadmap.

The goal of Week 0 was simple: **build the foundation required to start the actual Data Engineering work from Week 1.**

1. Git & GitHub

I installed Git and configured my Git identity.

Verify Git

```
git --version
```

Configure Git

```
git config --global user.name "My Name"  
git config --global user.email "my-email@example.com"
```

I then created my GitHub repository and connected my local project to it.

```
git init  
git remote add origin <repository-url>
```

I verified the connection with:

```
git remote -v
```

2. VS Code

I installed **Visual Studio Code** as my primary development environment.

I configured extensions for:

- Python
- Pylance
- Jupyter
- Docker
- Git
- SQL
- YAML

I kept the extensions limited to tools that I will actually use during the journey.

3. Python

I installed Python and verified the installation.

```
python --version
```

I also verified `pip`:

```
python -m pip --version
```

Then I upgraded pip:

```
python -m pip install --upgrade pip
```

4. Python Virtual Environment

I will use a separate virtual environment for each Python project.

I created one using:

```
python -m venv .venv
```

On Windows, I activated it with:

```
.venv\Scripts\activate
```

I verified that the environment was active:

```
python --version  
pip list
```

I also added `.venv/` to `.gitignore` so that the virtual environment is never pushed to GitHub.

5. Python Development Packages

I installed the basic packages required for my Data Engineering work:

```
pip install pandas numpy requests python-dotenv sqlalchemy
```

I saved the dependencies using:

```
pip freeze > requirements.txt
```

6. SQL & PostgreSQL Environment

I prepared my SQL development environment and PostgreSQL database.

I verified that I could connect to PostgreSQL and execute basic SQL queries successfully.

7. Docker

I installed Docker Desktop because I will use containers throughout the roadmap.

I verified Docker:

```
docker --version
```

I verified Docker Compose:

```
docker compose version
```

Finally, I tested the Docker Engine:

```
docker run hello-world
```

A successful response confirmed that Docker was working correctly.

8. Jupyter

I installed Jupyter for data exploration and experimentation.

```
pip install jupyter
```

I verified it by running:

```
jupyter notebook
```

9. Environment Variables

I configured my projects to keep credentials and API keys outside the source code.

For local development, I use a `.env` file:

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=my_database
DB_USER=my_user
DB_PASSWORD=my_password
API_KEY=my_api_key
```

I added `.env` to `.gitignore`:

```
.env  
.env.*
```



Database Preparation

Database preparation is also a part of Week 0.

I prepared the PostgreSQL database using SQL scripts and Python data-generation scripts.

The database setup follows a specific execution order. **The SQL files must be executed first, followed by the Python data-generation scripts.**

10. Database File Execution Order

This order matters.

Step 1 — Create Schemas

First, I connect to the PostgreSQL database:

```
100x
```

Then I run:

```
sql/01_create_schemas.sql
```

Step 2 — Create Tables

After the schemas are created, I run:

```
sql/02_create_tables.sql
```

Step 3 — Create Indexes

After the tables are created, I run:

```
sql/03_create_indexes.sql
```

I don't run views yet because they haven't been created at this stage.

11. Generate the Data

After completing the SQL setup, I generate the data using the Python scripts located in:

```
week-01/database/scripts/
```

The scripts are executed in the following order.

Step 1 — Generate Customers

```
python generate_customer.py
```

Step 2 — Generate Catalog

```
python generate_catalog.py
```

Step 3 — Generate Orders

```
python generate_orders.py
```

Step 4 — Generate Logistics

```
python generate_logistics.py
```

Step 5 — Validate the Database

Finally, I run:

```
python validate.py
```

The validation script confirms that the database and generated data have been prepared correctly.

12. Database Setup Flow

The complete execution flow is:

```
SQL
|
├─ 01_create_schemas.sql
|
├─ 02_create_tables.sql
|
└─ 03_create_indexes.sql
    |
    ▼
Python
|
├─ generate_customer.py
|
├─ generate_catalog.py
|
├─ generate_orders.py
|
├─ generate_logistics.py
|
└─ validate.py
```

This gives me a repeatable database setup that I can use for the upcoming Data Engineering work.

13. Project Structure

My 100x Data Engineer repository is organized into weekly modules:

```
100x-data-engineer/  
|  
├─ README.md  
├─ week-00/  
├─ week-01/  
├─ week-02/  
├─ projects/  
├─ exercises/  
├─ notes/  
└─ resources/
```

The database scripts used during Week 0 are maintained as part of the Week 1 database work.

14. Final Verification

I verified that my complete Week 0 foundation was working:

- ✓ Git installed and configured
 - ✓ GitHub repository created
 - ✓ VS Code configured
 - ✓ Python installed
 - ✓ Python virtual environment working
 - ✓ Required Python packages installed
 - ✓ PostgreSQL environment ready
 - ✓ Database schemas created
 - ✓ Database tables created
 - ✓ Database indexes created
 - ✓ Customer data generated
 - ✓ Catalog data generated
 - ✓ Order data generated
 - ✓ Logistics data generated
 - ✓ Database validation completed
 - ✓ Docker working
 - ✓ Docker Compose working
 - ✓ Jupyter working
 - ✓ Environment variables configured
 - ✓ Repository structure created
-

Week 0 Complete

My development environment and database foundation are ready.

Next → **Week 1**